

Experiment 7

Personal Productivity Assistant

Submission PDF

[Source folder on GitHub](#)

Prepared 2026-09-29 from the experiment materials on branch
`codex/complete-experiments`.

Source: [Ex07_Personal_Productivity_Assistant/README.md](#)

Exp-7: Prompt-Based Personal Productivity Assistant

Due: 25 Sep 2026 **Reference repo:** [RaajaThilahar/Ex.No.7](#)

Features

- **Task manager** - natural-language input ("Remind me to call mom at 6 PM"), priorities, daily summary
- **Smart scheduler** - events, overlap detection, free-slot finder
- **Wellness tips** - adapts to / feedback (stored preferences = memory)
- **LLM mode (optional)** - paste prompts/system_prompt.md into any LLM chat to use the same assistant conversationally; the CLI itself is fully offline.

Run

```
cd src && python3 assistant.py          # interactive CLI
python3 -m unittest discover -s tests   # tests (from this folder)
```

Commands: add <text>, list, summary, schedule <title> <HH:MM> <minutes>, free, tip, like, dislike, quit.

Files

File	Purpose
prompts/system_prompt.md	System prompt + per-task prompts for an LLM
src/assistant.py	CLI + core logic
src/memory.json	Auto-created preference store (git-ignored)
tests/	Unit tests
requirements.md	Core requirements
feedback.md	User feedback log + adaptations

Source: [Ex07_Personal_Productivity_Assistant/LAB_RECORD.md](#)

Lab Record: Exp-7: Personal Productivity Assistant

Field	Entry
Name / Reg. No.	_____
Date	_____
Repo	https://github.com/RHUDHRESH/PROMPT_ENGINEERING-

Aim

Build a prompt-based personal assistant with task, schedule and wellness features and preference memory.

Tools / Apparatus

Offline Python CLI; 4 unit tests passed in this session

Procedure

- Prompts written (see prompts/ or the folder README).
- Each prompt run on every tool; raw outputs saved under outputs/ (or the folder's equivalent).
- Results scored in feedback.md on: Feature coverage and adaptation to feedback.
- Findings summarised below.

Observations

See feedback.md. Add your own scores for any tool you ran yourself. Tools not run are left blank; do not fill them in from memory.

Result

The CLI accepts tasks, lists priorities, detects schedule overlaps, finds free slots and adapts wellness tips to feedback. Four unit tests passed.

Conclusion

A small rule-based program demonstrates the assistant workflow and persistent preferences without requiring an API. Its time parser handles a limited set of formats, so natural-language coverage remains narrower than a full LLM assistant.

Faculty signature: _____

Source: Ex07_Personal_Productivity_Assistant/feedback.md

User feedback log

All four sessions below are SIMULATED. No real users tested the assistant. Each session is a real run of the actual CLI (src/assistant.py, tip / like / dislike commands) driven by a scripted "persona" that gives feedback by a fixed rule. Tip categories are chosen randomly by the CLI, so the sequences are one sample, not an average, and re-running gives different sequences. The memory file was deleted between personas (Session 2 continues Session 1's memory on purpose). Run date 2026-09-29. The sequences of tips and feedback come from the runs; the "Observed" and "Adaptation" text is my reading of them.

A real transcript of a normal session (tasks, scheduling, one tip cycle) is in sample_session.txt.

#	Simulated user (rule)	What happened (observed)	Adaptation made / limitation seen
1	"Asha", day 1, fresh memory. Likes exercise tips, dislikes screen tips, ignores others. 8 tips	Sequence: hydration, hydration (no feedback), screen (disliked), then exercise x5 (each liked). Final memory: liked {exercise: 5}, disliked {screen: 1}	Weights become $1 + 2 * \text{likes}$: exercise weight 11 vs 1 for each other category. Only one dislike so screen is still eligible, since the code needs 2 dislikes to exclude a category. Nothing was changed in the code in response
2	"Asha", day 2, memory kept from Session 1. Likes exercise, no dislikes. 12 tips	Exercise was shown 11 of 12 times, hydration once, screen never (by chance; it was still eligible with weight 1). Memory: liked {exercise: 16}	Adaptation works but is very strong: likes never decay, so variety collapses. Suggested improvement (not implemented): cap the like weight or decay it, or add a small exploration floor

#	Simulated user (rule)	What happened (observed)	Adaptation made / limitation seen
3	"Ben", fresh memory. Likes hydration, dislikes exercise and screen. 12 tips	Sequence: screen (dis), screen (dis), exercise (dis), exercise (dis), then hydration x8 (liked). Final memory: liked {hydration: 8}, disliked {screen: 2, exercise: 2}	Clear adaptation: after 2 dislikes each, screen and exercise were excluded and only hydration appeared. This matches the requirement "never repeat a disliked tip category" only after 2 dislikes, not 1
4	"Cara", fresh memory. Dislikes every category. 12 tips	Tips kept coming from all three categories (hydration 5, screen 4, exercise 3), all disliked. Final memory: disliked {hydration: 5, screen: 4, exercise: 3}	Edge case: once every category has 2 or more dislikes, the code falls back to equal weights and shows everything again. Nothing better can be offered with only 3 categories; a real fix would be to add tip categories or offer to stop tips (not implemented)

Summary of what the simulation showed

- The like/dislike memory does change what is shown, and persists across runs through memory . json (Sessions 1 to 2).
- Threshold behaviour: 2 dislikes exclude a category; 1 dislike changes nothing.
- Weaknesses found: like weights grow without bound (Session 2), and the all-disliked fallback (Session 4).
- The system prompt in prompts/system_prompt.md says "never repeat a disliked tip category", which is stricter than the CLI's behaviour above.
- These findings come from simulated personas and single random runs. They are not evidence of how real users would react.

Source:

Ex07_Personal_Productivity_Assistant/prompts/system_prompt.md

System prompt

You are "Pace", a personal productivity assistant. Be concise and warm.
 You help with: (1) tasks and reminders, (2) scheduling, (3) wellness tips, (4) general questions.
 Always:

- Extract structured data as JSON when the user adds a task:

```
{"title":"","time":"HH:MM|null","priority":"high|medium|low"}
```
- Warn about overlapping events.
- Use the USER PREFERENCES block below to tailor tips; never repeat a disliked tip category.

USER PREFERENCES: {{memory_json}}

Task-extraction prompt

Extract a task from: "{{user_text}}". Return JSON only.

Wellness prompt

Suggest one wellness tip (<=20 words). Prefer categories {{liked}}, avoid {{disliked}}.

Source: Ex07_Personal_Productivity_Assistant/requirements.md

Core requirements

Accept tasks in natural language; extract title, time, priority.

Organise by priority then time; give daily summary of pending items.

Schedule events; warn on overlaps; list free slots (09:00-18:00).

Suggest wellness tips (hydration, exercise, screen breaks) and adapt to feedback.

Persist preferences between runs (memory).

Simple command-line interface.

Appendix: Source Code

Source: Ex07_Personal_Productivity_Assistant/src/assistant.py

```

"""Personal productivity assistant (offline rule-based core; LLM optional)."""
import json, os, re, random
from datetime import datetime

MEM = os.path.join(os.path.dirname(os.path.abspath(__file__)), "memory.json")
DAY_START, DAY_END = 9 * 60, 18 * 60
TIPS = {
    "hydration": ["Drink a glass of water now.", "Keep a bottle at your desk and refill it twice today."],
    "exercise": ["Take a 10-minute walk.", "Do 20 squats between tasks."],
    "screen": ["Look 20 feet away for 20 seconds (20-20-20 rule).", "Take a 5-minute screen break."],
}
HIGH = ("urgent", "asap", "important", "deadline", "today")
LOW = ("someday", "maybe", "whenever")

def to_min(hhmm):
    h, m = hhmm.split(":")
    return int(h) * 60 + int(m)

def fmt(mins):
    return f"{mins // 60:02d}:{mins % 60:02d}"

def parse_time(text):
    m = re.search(r"\b(\d{1,2})(?:\b(\d{2}))?\s*(am|pm)\b", text, re.I) or \
        re.search(r"\b(\d{1,2}):(\d{2})\b", text)
    if not m:
        return None
    h, mi = int(m.group(1)), int(m.group(2) or 0)
    if len(m.groups()) >= 3 and m.group(3):
        ap = m.group(3).lower()
        h = h % 12 + (12 if ap == "pm" else 0)
    return f"{h:02d}:{mi:02d}" if h < 24 and mi < 60 else None

def parse_task(text):
    low = text.lower()
    pr = "high" if any(w in low for w in HIGH) else "low" if any(w in low for w in LOW) else "medium"
    title = re.sub(r"^(remind me to|remember to|i need to)\s+", "", text.strip(), flags=re.I)
    t = parse_time(text)
    title = re.sub(r"\s\b(at|by)\s+\d{1,2}:(\d{2})?\s*(am|pm)?\b", "", title, flags=re.I).strip()
    return {"title": title, "time": t, "priority": pr, "done": False}

class Assistant:
    def __init__(self, path=MEM):
        self.path = path
        self.data = {"tasks": [], "events": [], "liked": {}, "disliked": {}, "last_tip": None}
        if os.path.exists(path):
            with open(path) as f:
                self.data.update(json.load(f))

    def save(self):
        with open(self.path, "w") as f:
            json.dump(self.data, f, indent=2)

    def add_task(self, text):
        t = parse_task(text)
        self.data["tasks"].append(t)
        self.save()
        return t

    def list_tasks(self):
        order = {"high": 0, "medium": 1, "low": 2}
        pend = [t for t in self.data["tasks"] if not t["done"]]
        return sorted(pend, key=lambda t: (order[t["priority"]], t["time"] or "99:99"))

    def summary(self):
        pend = self.list_tasks()
        if not pend:
            return "Nothing pending. Nice!"
        lines = [f"You have {len(pend)} pending task(s):"]

```

```

        lines += [f"- [{t['priority']}] {t['title']}" + (f" @ {t['time']}" if t["time"] else "") for t in
            pend]
        return "\n".join(lines)

def schedule(self, title, start, minutes):
    s, e = to_min(start), to_min(start) + int(minutes)
    clash = [ev for ev in self.data["events"] if s < ev["end"] and ev["start"] < e]
    self.data["events"].append({"title": title, "start": s, "end": e})
    self.save()
    if clash:
        return "Scheduled, but overlaps with: " + ", ".join(c["title"] for c in clash)
    return f"Scheduled {title} {fmt(s)}-{fmt(e)}."

def free_slots(self):
    cur, out = DAY_START, []
    for ev in sorted(self.data["events"], key=lambda x: x["start"]):
        if ev["start"] > cur:
            out.append((cur, ev["start"]))
            cur = max(cur, ev["end"])
    if cur < DAY_END:
        out.append((cur, DAY_END))
    return [f"{fmt(a)}-{fmt(b)}" for a, b in out]

def tip(self):
    weights = {c: 1 + 2 * self.data["liked"].get(c, 0) for c in TIPS if self.data["disliked"].get(c,
        0) < 2}
    if not weights:
        weights = {c: 1 for c in TIPS}
    cat = random.choices(list(weights), list(weights.values()))[0]
    self.data["last_tip"] = cat
    self.save()
    return f"[{cat}] {random.choice(TIPS[cat])}"

def feedback(self, positive):
    cat = self.data.get("last_tip")
    if not cat:
        return "Ask for a tip first."
    key = "liked" if positive else "disliked"
    self.data[key][cat] = self.data[key].get(cat, 0) + 1
    self.save()
    return f"Noted. I'll {'show more' if positive else 'show fewer'} {cat} tips."

def main():
    a = Assistant()
    print("Pace assistant. Commands: add, list, summary, schedule, free, tip, like, dislike, quit")
    while True:
        try:
            line = input("> ").strip()
        except EOFError:
            break
        cmd, _, rest = line.partition(" ")
        if cmd == "quit":
            break
        elif cmd == "add":
            print("Added:", a.add_task(rest))
        elif cmd == "list":
            print(a.summary())
        elif cmd == "summary":
            print(a.summary())
        elif cmd == "schedule":
            p = rest.rsplit(" ", 2)
            print(a.schedule(p[0], p[1], p[2]) if len(p) == 3 else "usage: schedule <title> <HH:MM>
                <minutes>")
        elif cmd == "free":
            print("Free:", ", ".join(a.free_slots()))
        elif cmd == "tip":
            print(a.tip())
        elif cmd in ("like", "dislike"):
            print(a.feedback(cmd == "like"))
        else:
            print("Unknown command.")

if __name__ == "__main__":
    main()

```

Source: Ex07_Personal_Productivity_Assistant/tests/test_assistant.py

```
import os, sys, tempfile, unittest
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
from assistant import Assistant, parse_task

class T(unittest.TestCase):
    def setUp(self):
        self.a = Assistant(os.path.join(tempfile.mkdtemp(), "m.json"))

    def test_parse(self):
        t = parse_task("Remind me to call mom at 6 PM")
        self.assertEqual((t["title"], t["time"], t["priority"]), ("call mom", "18:00", "medium"))
        self.assertEqual(parse_task("urgent: submit report by 09:30")["priority"], "high")

    def test_order(self):
        self.a.add_task("someday read a book")
        self.a.add_task("urgent pay bill")
        self.assertEqual(self.a.list_tasks()[0]["title"], "urgent pay bill")

    def test_overlap_and_free(self):
        self.a.schedule("Standup", "10:00", 30)
        self.assertIn("overlaps", self.a.schedule("Call", "10:15", 30))
        self.assertEqual(self.a.free_slots()[0], "09:00-10:00")

    def test_feedback_adapts(self):
        self.a.tip()
        cat = self.a.data["last_tip"]
        self.a.feedback(True)
        self.assertEqual(self.a.data["liked"][cat], 1)

if __name__ == "__main__":
    unittest.main()
```